

Cloud shared responsibility model

The cloud shared responsibility model is a security framework dividing duties between cloud service providers (CSPs like AWS, Azure) and customers. Providers handle security **of** the cloud (infrastructure, hardware), while customers handle security **in** the cloud (data, applications, configuration), varying by IaaS, PaaS, or SaaS models.

Key Aspects of the Model

- **Security of the Cloud (CSP):** The provider is responsible for protecting the infrastructure, physical data centers, host operating systems, and virtualization layers.
- **Security in the Cloud (Customer):** The customer is responsible for managing guest operating systems, application software, network traffic configuration, and, critically, protecting their data and managing access rights.
- **Purpose:** To prevent security gaps, ensure regulatory compliance, and define liability for data breaches.

Responsibility Shifts by Service Type

- **IaaS (Infrastructure as a Service):** Customer manages the OS, middleware, and applications; CSP manages physical hardware.
- **PaaS (Platform as a Service):** Customer manages applications and data; CSP manages OS and hardware.

- **SaaS (Software as a Service)**: Customer primarily manages access and data configuration; CSP manages almost all infrastructure and application components.

Customer Responsibilities (Common Misconceptions)

Even though the CSP secures the infrastructure, the customer is always responsible for: [\[1\]](#)

- **Data Protection**: Encryption and classification.
- **Identity & Access Management (IAM)**: Configuring who can access what.
- **Configuration**: Misconfiguration is a major cause of security breaches, which is a customer, not provider, duty. [\[1, 2, 3\]](#)

Understanding this model is essential, as failure to do so can leave data and applications vulnerable to attacks

Authentication, authorization in cloud computing

Authentication (AuthN) verifies user identity, while authorization (AuthZ) determines permitted actions, forming the foundation of cloud security. AuthN confirms users/systems are who they claim (via passwords/MFA), while AuthZ grants access to specific resources (using roles or policies) based on "least privilege".

Authentication in Cloud Computing (Who Are You?)

- **Purpose:** Confirms the identity of the user, service, or application.
- **Methods:** Includes usernames/passwords, multi-factor authentication (MFA/2FA), biometric verification, and single sign-on (SSO).
- **Key Concept:** In a [Cloud Environment](#), MFA is considered mandatory for robust security.
- **Principals:** Identifies both human users and service accounts (automated systems). [[1](#), [2](#), [3](#), [4](#)]

Authorization in Cloud Computing (What Can You Do?)

- **Purpose:** Grants permission to an authenticated entity to access specific data or perform actions.
- **Key Components:**
 - **Access Control:** Determines who gets to use what.
 - **Roles (RBAC):** Assigns permissions based on roles (e.g., administrator, user) rather than individuals.
 - **Policies:** Rulebooks defining when someone can access data (e.g., based on time or IP).
 - **Token-based Access:** Uses digital tokens (e.g., OAuth 2.0) to prove authority after login. [[1](#), [2](#), [3](#)]

Key Differences

Feature	Authentication (AuthN)	Authorization
---------	------------------------	---------------

		(AuthZ)
Focus	Identity Verification	Permission Granting
Question	"Who are you?"	"What can you do?"
Process	Username/Password, MFA, SSO	Roles, Policies (RBAC/ABAC)
Sequence	Occurs First	Occurs Second

Common Cloud Security Frameworks

- **Zero Trust:** Never trust, always verify—crucial for securing cloud resources.
- **AAA (Authentication, Authorization, Accounting):** Framework that verifies, permits, and tracks user actions.
- **IAM (Identity and Access Management):** Standard cloud service to manage both authentication and authorization.

Authentication Vs Authorization

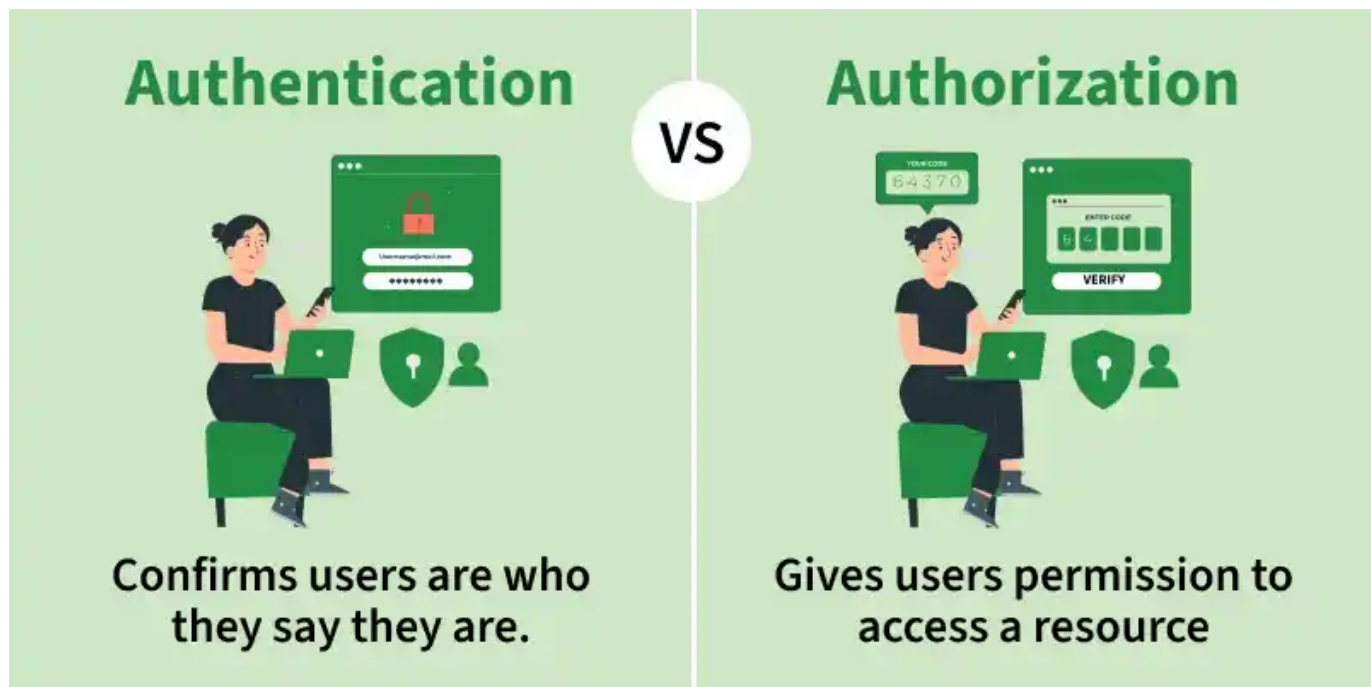
Authentication and Authorization are key security steps that ensure only the right users access a system and can perform permitted actions. Authentication verifies who you are, and Authorization determines what you can do.

- **Authentication:** Confirms the user's identity.

- Authorization: Determines what the verified user is allowed to access or perform.

Working of Authentication and Authorization

This explains how users are first verified and then given the appropriate access:



Authentication

Authentication is the process of verifying the identity of a user or system. It ensures that the user is legitimate by validating credentials like passwords, OTPs, or biometrics.

Working

The authentication process verifies the identity of a user before granting access to the system.

- User enters credentials (password, OTP, biometrics)
- System verifies the credentials

- If valid, the user is successfully authenticated

Authorization

Authorization determines the access rights and permissions of an authenticated user. It decides what resources the user can access and what actions they are allowed to perform.

Working

The system uses authentication and authorization processes to control access and ensure security.

- System checks the user's roles or permissions
- Grants or denies access to resources
- Ensures the user can perform only allowed actions

Difference Between Authentication and Authorization

Here are some main differences between Authentication and Authorization:

Authentication	Authorization
Verifies who the user is	Determines what the user can access
Performed before authorization	Happens after authentication
Requires login details	Requires user roles,

(username, password, biometrics)	privileges, or access levels
Determines if the user is valid	Determines what permissions the valid user has
Uses ID Tokens	Uses Access Tokens
Governed by OpenID Connect (OIDC)	Governed by OAuth 2.0
Credentials can be changed by the user	Permissions can only be changed by the system owner
Visible to the user (entering credentials)	Not visible to the user (handled in the background)
Examples: Password, OTP, fingerprint, face recognition	Examples: Admin rights, read/write access, role-based permissions

AWS: Identity and Access Management

Identity and Access Management (IAM) ensures secure control over who can access cloud resources and what actions they are allowed to perform.

- Protects systems from unauthorized access and credential misuse
- Controls user identities, roles, and permissions centrally
- Follows the principle of least privilege for stronger security
- Reduces risks from weak, stolen, or shared passwords
- Essential for secure and scalable cloud environments

Identity and Access Management (IAM)

IAM is a security framework that manages authentication (who can sign in) and authorization (what they are allowed to do) across systems and resources.

Amazon Web Services Identity and Access Management (IAM) is a web service that securely controls access to AWS resources and acts as the backbone of account-level security.

- Controls user identities and permissions centrally
- Prevents unauthorized access to cloud services

- Supports secure scaling of users, applications, and services

Core Components of AWS IAM

IAM is built on several core components that represent "who" can access your account.

1. IAM Users

IAM Users represent individual people or applications requiring long-term access to AWS.

- Have permanent credentials (passwords and access keys)
- Used for console, CLI, or API access
- Best managed through groups, not individually

2. IAM Groups

IAM Groups are collections of users that simplify permission management.

- Permissions are assigned once at the group level
- Users inherit permissions automatically
- Reduces administrative errors and improves scalability

3. IAM Roles

IAM Roles provide temporary permissions to trusted entities without using long-term credentials.

- No passwords or access keys
- Uses temporary, auto-rotated credentials
- Ideal for AWS services and cross-account access

- Most secure method for application permissions

Feature	IAM User	IAM Group	IAM Role
Purpose	Represents a specific person or application with long-term credentials.	A logical container to simplify permission management across users.	A temporary identity assumed by trusted entities for specific tasks.
Credentials	Permanent (password, access keys).	None; not an identity.	Temporary, automatically rotated credentials.
Use for Humans	Rarely used for direct access; reserved for administrative or break-glass scenarios.	Assign common permissions (e.g., Developers, Auditors).	Grant short-term, elevated, or cross-account access.
Use for Services	Not recommended due to long-term	Not applicable.	Best practice for service-to-service

	key risks.		access (e.g., EC2 → S3).
--	------------	--	--------------------------------

Identity and Access Management Working

Identity and Access Management (IAM) works by clearly defining who can access AWS, what they can access, and under what conditions. Its working can be understood in the following flow:

1. Identify the User or Entity (Who)

IAM first identifies who is requesting access. This could be:

- An IAM user (human)
 - An application or service
 - An AWS service assuming a role
- Authentication is done using credentials such as passwords, access keys, or MFA.

2. Authenticate the Request

IAM verifies the identity using authentication mechanisms like:

- Username and password
 - Access keys
 - Multi-Factor Authentication (MFA)
- Only authenticated identities can proceed further.

3. Authorize Using Policies (Can Access)

After authentication, IAM checks **policies** attached to the user, group, or role. These policies define:

- Allowed or denied actions (e.g., s3:GetObject)
- Specific resources (e.g., a particular S3 bucket)
- Conditions (time, IP address, MFA usage)

4. Evaluate Permissions

IAM evaluates permissions in this order:

- **Explicit Deny** → always blocks access
- **Explicit Allow** → grants access
- **Implicit Deny** → default if no allow exists

Access is granted only if an explicit allow exists and no deny overrides it.

5. Grant Access to Resources (What)

If authorization succeeds, IAM allows the user or service to perform the requested action on the specified AWS resource such as:

- EC2 instances
- S3 buckets
- RDS databases

6. Temporary Access via Roles (Optional but Recommended)

For applications and services, IAM roles provide temporary credentials instead of long-term keys, improving security and reducing credential misuse.



AWS Identity and Access Management

Apply fine-grained permissions to AWS services and resources

Who



Workforce users with AWS SSO and workloads with IAM roles

Can access



Permissions with IAM policies

What



Resources within your AWS organization